```python
import pandas as pd
# นำเข้าไลบรารี pandas ใช้จัดการข้อมูลแบบตาราง (DataFrame)
# เช่น อ่านไฟล์ CSV, จัดเรียงข้อมูล, เลือกคอลัมน์

import numpy as np
# นำเข้าไลบรารี numpy ใช้คำนวณทางคณิตศาสตร์และจัดการข้อมูลแบบอาร์เรย์
# เช่น การคำนวณตัวเลขจำนวนมาก, เมทริกซ์

import seaborn as sns
# นำเข้าไลบรารี seaborn ใช้สร้างกราฟเชิงสถิติที่สวยงาม
# มักใช้คู่กับ pandas เช่น pairplot, heatmap

import matplotlib.pyplot as plt
# นำเข้า matplotlib.pyplot ใช้สร้างกราฟพื้นฐาน
# เช่น กราฟเส้น กราฟแท่ง กราฟจุด

from sklearn.preprocessing import StandardScaler
# นำเข้า StandardScaler ใช้ปรับสเกลข้อมูล
# ทำให้ข้อมูลมีค่าเฉลี่ย = 0 และส่วนเบี่ยงเบนมาตรฐาน = 1
# สำคัญมากสำหรับโมเดล KNN

from sklearn.neighbors import KNeighborsClassifier
# นำเข้าโมเดล K-Nearest Neighbors (KNN)
# ใช้สำหรับงานจำแนกประเภท (Classification)

from sklearn.metrics import accuracy_score
# นำเข้า accuracy_score ใช้ประเมินความถูกต้องของโมเดล
# เปรียบเทียบค่าที่โมเดลทำนายกับค่าจริง
```

```python
df_train = pd.read_csv('train.csv')
# อ่านไฟล์ train.csv แล้วเก็บข้อมูลไว้ในตัวแปร df_train
# ข้อมูลจะอยู่ในรูปแบบตาราง (DataFrame)

df_train.head()
# แสดงข้อมูล 5 แถวแรกของ DataFrame
# ใช้ดูตัวอย่างข้อมูลเบื้องต้นว่ามีคอลัมน์อะไรบ้าง

df_train.info()
# แสดงข้อมูลสรุปของ DataFrame
# เช่น จำนวนแถว จำนวนคอลัมน์ ชื่อคอลัมน์
# ชนิดข้อมูล (int, float, object)
# และจำนวนข้อมูลที่ไม่เป็นค่าว่าง (non-null)

df_train.describe()
# แสดงค่าสถิติพื้นฐานของข้อมูลเชิงตัวเลข
# เช่น count, mean, std, min, max, 25%, 50%, 75%
# ใช้ดูการกระจายของข้อมูล

df_train.isnull().sum()
# ตรวจสอบค่าที่หายไป (Missing Values)
# isnull() จะตรวจว่าค่าไหนเป็นค่าว่าง (NaN)
# sum() จะนับจำนวนค่าว่างในแต่ละคอลัมน์

df_train.shape
# ใช้ตรวจสอบขนาดของ DataFrame
# ผลลัพธ์จะเป็น (จำนวนแถว, จำนวนคอลัมน์)
```

```python
sns.pairplot(
    df_train,
    # กำหนด DataFrame ที่ใช้ในการสร้างกราฟ (df_tr

    vars=['Age', 'Fare', 'Pclass', 'SibSp'],
    # เลือกเฉพาะคอลัมน์ที่ต้องการนำมาวิเคราะห์และแสดง
    # Age = อายุ
    # Fare = ค่าโดยสาร
    # Pclass = ชั้นโดยสาร
    # SibSp = จำนวนพี่น้อง/คู่สมรสบนเรือ

    hue='Survived',
    # ใช้คอลัมน์ Survived แยกสีของข้อมูล
    # เช่น รอดชีวิต (1) และไม่รอดชีวิต (0)

    markers=['o', 's'],
    # กำหนดรูปแบบสัญลักษณ์ของจุดในกราฟ
    # 'o' = วงกลม, 's' = สี่เหลี่ยม
    # ใช้แทนค่าที่ต่างกันของ Survived

    plot_kws={'alpha': 0.6}
    # กำหนดความโปร่งใสของจุดข้อมูล
    # ค่า alpha อยู่ระหว่าง 0-1 (ยิ่งน้อยยิ่งโปร่งใส)
)

plt.show()
# แสดงกราฟทั้งหมดออกมาบนหน้าจอ
```

## ขั้นตอนที่ 4 : กำหนด Features และ Target

```
136]: y = df_train['Survived']
      df_train['Age'].fillna(df_train['Age'].median(), inplace=True)
      df_train['Embarked'].fillna(df_train['Embarked'].mode()[0], inplace=True)
      df_train.isnull().sum()
      # กำหนดตัวแปร y ให้เก็บคอลัมน์ Survived
      # ใช้เป็นตัวแปรเป้าหมาย (Target / Label)
      # สำหรับการสร้างโมเดล Machine Learning
```

```python
# แปลงเพศเป็นตัวเลข
df_train['Sex'] = df_train['Sex'].map({'male': 0, 'female': 1})
# แปลงข้อมูลเพศจากข้อความเป็นตัวเลข
# male = 0, female = 1
# เพื่อให้โมเดล Machine Learning สามารถนำไปคำนวณได้

# One-Hot Encoding สำหรับ Embarked
df_train = pd.get_dummies(df_train, columns=['Embarked'], drop_first=True)
# แปลงข้อมูล Embarked (ท่าเรือที่ขึ้น) ซึ่งเป็นข้อมูลเชิงกลุ่ม (categorical)
# ให้เป็นคอลัมน์ตัวเลขหลายคอลัมน์
# drop_first=True เพื่อลดปัญหา multicollinearity
# จะเหลือเฉพาะ Embarked_Q และ Embarked_S
```

```python
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare',
            'Embarked_Q', 'Embarked_S']
# กำหนดรายชื่อคอลัมน์ที่ใช้เป็นตัวแปรอิสระ (Features)
# เป็นข้อมูลที่ใช้ให้โมเดลเรียนรู้

X = df_train[features]
# สร้างตัวแปร X เพื่อเก็บข้อมูล Features
# X จะเป็นข้อมูลนำเข้า (Input) ของโมเดล

y = df_train['Survived']
# กำหนดตัวแปร y เป็นตัวแปรเป้าหมาย (Target / Label)
# ใช้บอกผลลัพธ์ว่าผู้โดยสารรอดชีวิตหรือไม่
```

# ขั้นตอนที่ 5 แบ่งข้อมูลออกเป็น Train และ Test

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    # X = ตัวแปรอิสระ (Features)
    # y = ตัวแปรเป้าหมาย (Target)

    test_size=0.2,
    # กำหนดสัดส่วนข้อมูลสำหรับทดสอบโมเดล = 20%
    # ส่วนที่เหลือ 80% จะใช้สำหรับฝึกโมเดล

    random_state=42
    # กำหนดค่า seed เพื่อให้การสุ่มแบ่งข้อมูลเหมือนเดิมทุกครั้ง
    # ช่วยให้ผลลัพธ์สามารถทำซ้ำได้ (reproducible)
)
```

# ตรวจสอบขนาดของข้อมูลที่แบ่งแล้ว

```
print(X_train.shape)

print(X_test.shape)

print(y_train.shape)

print(y_test.shape)
```

## ขั้นตอนที่ 6 : การปรับขนาดข้อมูล (Scaling)

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

## ขั้นตอนที่ 7 : การสร้างและฝึกโมเดล (Model Building & Training)

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
knn_model.fit(X_train, y_train) # ฝึกโมเดลด้วยข้อมูลชุดฝึก (Model Training)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

# ขั้นตอนที่ 8 : การทำนายผลและการประเมินโมเดล

```
y_pred_knn = knn_model.predict(X_test)
```

```
result = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred_knn
})
result.head(10)
```

| | Actual | Predicted |
|---|---|---|
| **0** | 1 | 0 |

# ขั้นตอนที่ 9 : การประเมินประสิทธิภาพด้วยค่าความถูกต้อง (Accuracy)

•[157]:
```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("KNN Accuracy:", accuracy_knn)
print("KNN Accuracy:", accuracy_knn * 100, "%")
```

```
KNN Accuracy: 0.8044692737430168
KNN Accuracy: 80.44692737430168 %
```

•[159]:
```
# ค่า Accuracy ของแต่ละโมเดล
models = ['KNN', 'Logistic Regression']
accuracies = [0.8044692737430168, 0.8100558659217877]
plt.figure()
plt.bar(models, accuracies, color=['skyblue', 'salmon'])
plt.ylim(0, 1)
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy Score')
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.02, f"{v:.3f}", ha='center')
plt.show()
# แสดงกราฟบนหน้าจอ
```

# ทำการทดสอบแบบคีย์ข้อมูลของผู้โดยสารลงเ

```
import numpy as np

# รับข้อมูลจากผู้ใช้
pclass = int(input("กรุณากรอกชั้นโดยสาร (1, 2, 3): "))
sex = int(input("เพศ (ชาย=0, หญิง=1): "))
age = float(input("อายุ: "))
sibsp = int(input("จำนวนพี่น้อง/คู่สมรส (SibSp): "))
parch = int(input("จำนวนพ่อแม่/บุตร (Parch): "))
fare = float(input("ค่าโดยสาร (Fare): "))
embarked_q = int(input("ขึ้นเรือที่ Q (ใช่=1, ไม่ใช่=0): "))
embarked_s = int(input("ขึ้นเรือที่ S (ใช่=1, ไม่ใช่=0): "))

new_passenger = np.array([[
    pclass,
    sex,
    age,
    sibsp,
    parch,
    fare,
    embarked_q,
    embarked_s
]])
new_passenger_scaled = scaler.transform(new_passenger)
prediction = knn_model.predict(new_passenger_scaled)
```

```
if prediction[0] == 1:
    print("ผลการทำนาย: ผู้โดยสารมีโอกาสรอดชีวิต")
else:
    print("ผลการทำนาย: ผู้โดยสารมีโอกาสไม่รอดชีวิต")
```

ผลการทำนาย: ผู้โดยสารมีโอกาสไม่รอดชีวิต